



T-SQL Cursors with WHILE Loop in AdventureWorks

Overview

Cursors in T-SQL allow row-by-row processing of query results. While generally less efficient than set-based operations, they're useful for specific scenarios requiring iterative processing. This note covers cursor usage with WHILE loops, @@FETCH_STATUS, and practical examples using the AdventureWorks database.

Key Concepts

- **Cursor:** A database object that points to a result set and allows row-by-row navigation.
- **@@FETCH_STATUS:** A system function that returns the status of the last cursor FETCH operation:
 - 0 : Fetch successful
 - -1 : Fetch failed or row is beyond result set
 - -2 : Fetched row is missing (e.g., deleted)
- **WHILE Loop:** Used with cursors to iterate through rows until @@FETCH_STATUS indicates no more rows.

Basic Cursor Structure

1. **Declare Cursor:** Define the cursor and its query.
2. **Open Cursor:** Initialize the cursor for reading.
3. **Fetch Rows:** Use FETCH to retrieve rows in a WHILE loop.
4. **Close and Deallocate:** Clean up cursor resources.

Example 1: Basic Cursor with AdventureWorks

This example iterates through employee names in the `HumanResources.Employee` table.

```

-- Declare variables
DECLARE @FirstName NVARCHAR(50), @LastName NVARCHAR(50);

-- Declare cursor
DECLARE employee_cursor CURSOR FOR
SELECT FirstName, LastName
FROM Person.Person
WHERE BusinessEntityID IN (SELECT BusinessEntityID FROM HumanResources.Employee);

-- Open cursor
OPEN employee_cursor;

-- Fetch first row
FETCH NEXT FROM employee_cursor INTO @FirstName, @LastName;

-- Loop through rows
WHILE @@FETCH_STATUS = 0
BEGIN
    -- Process each row
    PRINT 'Employee: ' + @FirstName + ' ' + @LastName;

    -- Fetch next row
    FETCH NEXT FROM employee_cursor INTO @FirstName, @LastName;
END;

-- Clean up
CLOSE employee_cursor;
DEALLOCATE employee_cursor;

```

Example 2: Updating Records with Cursor

This example updates employee vacation hours based on their job title.

```

-- Declare variables
DECLARE @BusinessEntityID INT, @JobTitle NVARCHAR(50), @VacationHours INT;

-- Declare cursor with specific options
DECLARE update_cursor CURSOR LOCAL FAST_FORWARD FOR
SELECT BusinessEntityID, JobTitle, VacationHours
FROM HumanResources.Employee
WHERE JobTitle LIKE '%Manager%';

-- Open cursor
OPEN update_cursor;

-- Fetch first row
FETCH NEXT FROM update_cursor INTO @BusinessEntityID, @JobTitle, @VacationHours;

-- Loop through rows
WHILE @@FETCH_STATUS = 0
BEGIN
    -- Update vacation hours for managers
    UPDATE HumanResources.Employee
    SET VacationHours = VacationHours + 10
    WHERE BusinessEntityID = @BusinessEntityID;

    PRINT 'Updated ' + @JobTitle + ': New Vacation Hours = ' + CAST(@VacationHours + 10 AS NVARCHAR(50));

    -- Fetch next row
    FETCH NEXT FROM update_cursor INTO @BusinessEntityID, @JobTitle, @VacationHours;
END;

-- Clean up
CLOSE update_cursor;
DEALLOCATE update_cursor;

```

Cursor Options

- **LOCAL:** Cursor is only available in the current scope.
- **GLOBAL:** Cursor is available to the connection.
- **FAST_FORWARD:** Optimizes for forward-only, read-only cursors.

- **SCROLL**: Allows backward and forward navigation.
- **READ_ONLY**: Prevents updates through the cursor.

Best Practices

1. **Use Set-Based Operations When Possible**: Cursors are less efficient than set-based queries.
2. **Use FAST_FORWARD**: For simple, forward-only operations to improve performance.
3. **Close and Deallocate**: Always clean up cursors to free resources.
4. **Limit Scope**: Use LOCAL cursors unless GLOBAL is required.
5. **Check @@FETCH_STATUS**: Always verify fetch status to avoid infinite loops.

Common Pitfalls

- **Performance Issues**: Cursors can be slow for large datasets.
- **Resource Leaks**: Failing to close/deallocate cursors.
- **Infinite Loops**: Not properly handling @@FETCH_STATUS.

AdventureWorks-Specific Notes

- The AdventureWorks database (e.g., AdventureWorks2019) provides tables like `Person.Person`, `HumanResources.Employee`, and `Sales.SalesOrderHeader` for cursor operations.
- Ensure you have appropriate permissions to read/update tables.
- Use meaningful queries to limit cursor result sets (e.g., filter by BusinessEntityID or date ranges).